

システム開発報告書

タクトーン

IMU ジェスチャで奏でる Arduino オーケストラ

情報工学科 2 年

25G1053

齋藤 翔太

チーム 23

片岡 聖 (24G1038) 地曳 賢人 (24G1075)

御代川 稜 (24G1131) 梅澤 颯太 (25G1021)

塩澤 匠生 (25G1065)

2026 年 7 月 16 日

1 はじめに

1.1 社会的背景

文化庁の文化芸術推進基本計画では、文化芸術は人の創造性や豊かな人間性を育むだけでなく、人と人とのつながりを強めるものと位置付けられている [9]。また、同計画は、デジタル技術によって表現形態や文化芸術体験が多様化していることを示す一方、演者と観客が同じ場で共有する直接的な体験や一体感の重要性も指摘している。したがって、デジタル技術を使いながら、身体動作と音が結び付く参加型の音楽体験を実現することには社会的意義がある。

総務省統計局の令和3年社会生活基本調査では、過去1年間にスマートフォンなどで音楽を鑑賞した人の割合が50%を超える一方、楽器を演奏した人は約10%であった [10]。この差だけから楽器演奏への障壁の原因を特定することはできないが、音楽を聴く機会に比べて、自ら演奏へ参加する機会は限られていることが分かる。さらに、指揮者付きオーケストラは一人では成立せず、指揮を練習するには複数の演奏者と場所を確保する必要があるため、初心者が日常的に体験することは難しい。

そこで本チームは、実際の演奏者を集めなくても、指揮棒を振る身体動作によって複数の楽器を動かせるシステムを提案した。単に曲を再生するのではなく、利用者のテンポが演奏へ反映されることで、指揮者として合奏へ能動的に参加する感覚を提供することを目指した。

1.2 関連技術と本システムの位置付け

テンポを扱う既存機器には、ボタンを叩く間隔からBPMを求めるタップ式BPMカウンタがあるが、指揮棒を振る身体動作そのものは演奏に反映されない。また、ヤマハのSYNCROOMは、ネットワークの遅延と揺らぎを小さな受信バッファで吸収し、離れた場所にいる演奏者のリアルタイム合奏を支援する [11]。しかし、SYNCROOMは人間の演奏者同士を接続するシステムであり、一人で指揮練習を成立させるものではない。

加速度センサなどから指揮動作を認識し、MIDI演奏を制御する指揮シミュレータは以前から研究されている [12]。本システムは、この考え方を複数の実機ノードによる合奏へ展開し、指揮ジェスチャの検出、5台への制御情報の一斉配信、各楽器の楽譜進行、PC上での音色合成までを一つの体験として統合する点に特徴がある。

1.3 技術的課題

第1の課題は、IMUの加速度と角速度から、利用者が意図した拍、テンポ、強弱を安定して抽出することである。手振りには個人差やセンサノイズがあり、単純なしきい値では誤検出と検出漏れが生じる。さらに、指揮動作の研究では、熟達者の打点は実際の拍点より先行する傾向が報告されている [13]。そのため、振り下ろしを検出した瞬間に音を鳴らすだけでなく、予備動作から発音予定時刻を予測し、指揮らしい時間関係を設計する必要がある。

第2の課題は、無線通信を介した複数ノードの同期である。合奏では、各演奏者が音響・視覚的な手掛かりを使ってタイミングを相互調整している [14] が、マイコンは共通の時刻と拍情報を明示的に共有しなければならない。WiFi 通信には到着時刻のばらつきやパケット欠落があるため、受信直後に発音すると楽器ごとの差がそのまま同期誤差になる。そこで本システムでは、SYNCRROOM と同様に通信の揺らぎを吸収するという考え方を参照しつつ、複数のマイコンへ同じ発音時刻を指示できる時計同期と発音予約を採用した。これに加えて、周期的な処理停止による外れ値も抑える必要がある。

第3の課題は、可変テンポの下で楽譜を正しく進めることである。各楽器が独立したタイマだけで演奏すると、時計のずれや受信漏れによって曲中の位置がずれる。このため、各ノードが拍番号から楽譜位置を毎回計算し、通信を一度受け取り損ねても次の拍で正しい位置へ復帰できる構造が必要となる。

第4の課題は、実機の制約内で楽器らしい音とエンターテインメント性を両立することである。Arduino は演奏制御と楽譜進行を担当し、音響処理は PC へ分離する必要がある。PC 側では、低い処理負荷で複数音を同時に再生しながら、楽器ごとの倍音構造、エンベロープ、原音サンプルを再現し、指揮動作に対する聴覚的な応答を分かりやすく提示することが求められる。

1.4 本プロジェクトの目的と本報告書の構成

以上の背景と課題から、本プロジェクトでは、複数の実演奏者を集めなくても、利用者が指揮棒を振る身体動作によって合奏を制御できるシステムの実現を目的とした。具体的には、ESP32-S3 と IMU を用いた指揮者ノードで拍とテンポを検出し、Arduino UNO R4 WiFi を用いた5台の楽器ノードへ制御情報を配信し、PC 上の Processing で各楽器の音を再生する「タクトーン」を制作した。授業資料と報告書評価ループリックで示された評価観点 [1, 2] に基づき、拍検出、楽譜再生、楽器間同期、テンポ追従、通信信頼性、音階誤差、およびエンターテインメント性を評価する。

筆者は、楽譜データの作成、音階生成ロジックの検討、および Processing 上での4声輪唱確認用スケッチの作成を主に担当した。本報告書では、はじめに要求とシステム全体の設計方針を示し、次に筆者の担当範囲と実装内容を説明する。その後、実機評価と発表会アンケートの結果をもとに有効性と限界を考察し、最後に成果と今後の課題をまとめる。なお、音階誤差は合成処理を Python で再現して評価したが、実際の音響出力を録音して周波数分析する評価は未実施である。また、観客アンケートはエンターテインメント性を評価したものであり、音楽経験の少ない利用者による操作性は評価していない。

2 要求と設計方針

本プロジェクトの目的は、複数の実演奏者を集めなくても、指揮者役の利用者が複数楽器による合奏を制御できる体験を実現することである。チーム内では、ユーザが指揮を振るテンポを変えたときに演奏速度も変わることを中心的なゴールとした。また、同期の成否を判断する指標として、楽器間の同期誤差 20 ms 以内を目安にした。

表 1 主要要求と対応方針

要求	内容	対応方針
同期演奏	5 台の楽器で演奏タイミングを合わせる	指揮者ノードから楽器ノードへ UDP で拍情報を送る
可変テンポ 輪唱	指揮の速さに演奏を追従させる 複数パートがずれて同じ主旋律を演奏する	IMU の動きから拍とテンポを推定する 各パートに開始拍を設定し、同じ楽譜を時間差で使う
楽器音らしさ	目的の楽器に聞こえる音を鳴らす	Processing で倍音構造, ADSR, 原音サンプルを用いて音色を作る
エンターテインメント性	見て分かる演奏体験にする	指揮を振る動作そのものを演奏体験の中心に置く

表 1 に、本システムで重視した要求と対応方針を示す。

3 システム全体構成

最終システムは、指揮者ノード 1 台、楽器ノード 5 台、各楽器に対応する Processing PC 5 台から構成される [3]。図 1 に、構成要素と主なデータの流れを示す。

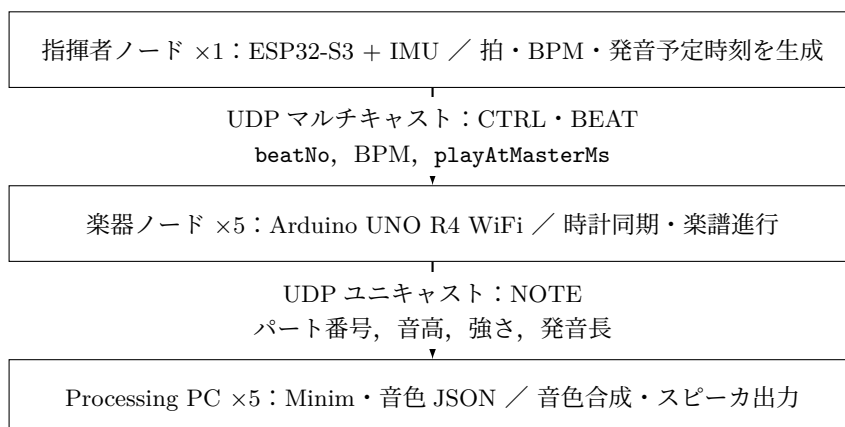


図 1 最終システムの全体構成とデータフロー

指揮者ノードは IMU の加速度と角速度から拍と BPM を推定し、拍番号 `beatNo` と発音予定時刻 `playAtMasterMs` を含む BEAT パケットを 5 台へ一斉配信する。各楽器ノードは CTRL パケットによって指揮者側のマスタ時計との差を推定し、共通の `playAtMasterMs` まで待機して発音処理を行う。これにより、BEAT パケットの到着時刻が楽器ごとに異なっても、発音時刻をそろえられる。

楽器ノードは、受信した絶対拍番号からパートの開始拍を引いて相対拍番号を求め、`ScoreEvent` 配列から該当する音符を選ぶ。したがって、パケットを一度受信し損ねても、次の BEAT を受信した時点で正しい楽譜位置へ復帰できる。選択した音符は NOTE パケットとして対応する PC へ送り、Processing と Minim が音色 JSON の倍音構造やエンベロープ、原音サンプルを用いて再生する。

表 2 最終システムの楽器構成

パート・ノード	音色	開始拍	音域・役割
主旋律 1 (node_02)	トランペット	0 拍	C5～A5
主旋律 2 (node_03)	ホルン	8 拍	C4～A4
主旋律 3 (node_04)	フルート	16 拍	C5～A5
主旋律 4 (node_05)	オルガン	24 拍	C3～A3
伴奏 (node_06)	ドラム	0 拍	56 拍のリズム伴奏

通信方式には、1 対多の同時配信が可能で再送待ちを生じない UDP を採用した。到達保証がない点は、BEAT の複数回送信と絶対拍番号による復帰で補い、到着時刻のばらつきは時計同期と発音予約で吸収する設計とした。また、Arduino 側は入力、ロジック、出力を分離し、共通処理とパート固有の楽譜・開始拍を分けることで、5 台のコード差分を小さくした。

4 個人担当範囲

筆者は、主に楽譜データと音色確認用 Processing スケッチを担当した [4]。具体的には、「かえるのうた」の楽譜構造を Arduino 側で扱える ScoreEvent へ変換し、4 声の輪唱として成立するように開始拍、音域、音色、音量バランスを設計した。また、複数パートを単独または一斉に再生できる Processing 確認用スケッチを作り、実機結合前に楽譜、輪唱の入り、ドラム伴奏、音色 JSON を確認できるようにした。本番用 Processing アプリの音響再生は梅澤が担当しており、筆者の確認用スケッチは、楽譜と音色設定を独立して検証するための環境という位置付けである。

最終システムでは、トランペット、ホルン、フルート、オルガンの 4 声に、ドラムを加えた 5 楽器ノード構成を採用した。これにより、金管の中音域、高音域の旋律、オルガンによる低音域の支えを分けて確認できるようにした。

表 2 に、最終システムの楽器構成を示す。4 つの主旋律パートは同じ基準譜面を使い、開始拍とオクターブ移調を変えることで輪唱を構成している。

5 実装内容

5.1 楽譜データの設計判断

初期案ではパートごとに別の旋律や低音伴奏を持たせていたが、音符修正のたびに複数配列を更新する必要があり、パート間で内容がずれるおそれがあった。そこで、32 拍の基準譜面 MELODY_SCORE を 4 声で共用し、パートごとの差を開始拍、オクターブ移調、音量、音色ファイルに限定した。「かえるのうた」の最初のまとまりが 8 拍であるため、開始拍を 0, 8, 16, 24 拍とし、前の声部が一つのまとまりを演奏した後に次の声部が入る輪唱とした。最後の声部は 24 拍目から 32 拍の旋律を演奏するため、伴奏ドラムは曲全体を覆う $24 + 32 = 56$ 拍とした。

楽譜データは、通常の進行を追いやすように 1 拍を 1 要素として扱う構造にした。Arduino

側では `ScoreEvent` 構造体を用意し、拍番号、MIDI ノート番号、ベロシティ、発音長、休符フラグを保持する。発音長は 1 拍を 256 とする固定小数点形式で保持し、終盤の半拍音符には 128 を指定する。また、24～27 拍目にある 1 拍内の 2 音を表現するために、`subNote`, `subOffsetQ8`, `subDurationQ8` などの補助フィールドを用意した。

配列全体を半拍単位に変更する方法も考えられるが、通常部分の要素数が倍になり、Arduino 側の楽譜進行も複雑になる。補助音方式にすることで、1 拍単位の読みやすさを保ちながら、必要な箇所だけ拍途中の音を表現できるようにした。

5.2 音色と再生確認の設計判断

Processing 確認用スケッチでは、音色 JSON を読み込み、倍音比、倍音振幅、エンベロープをもとに音を合成する。金管系の音色では複数の正弦波を重ね、ADSR で音の立ち上がりと減衰を制御した。ドラムはキック、スネア、ハイハット、クラッシュを用意し、4 分の 4 拍子として 1・3 拍目にキック、2・4 拍目にスネアを配置した。各声部が入る 0, 8, 16, 24 拍目ではクラッシュとキックを重ね、どの位置で声部が増えたかを聴覚的に把握しやすくした。

フルートについては、倍音合成だけでは自然な質感を出しにくかったため、`tone_sample` を含む音色 JSON を用いた。スケッチ側では `ToneSampleUGen` を実装し、原音のループ区間を線形補間しながら読み出し、基準音からの半音差に応じて再生速度を変える方式を採用した。これにより、原音の質感を残しながら楽譜上の複数音高を再生できる。

金管の立ち上がりがドラムより遅く聞こえる問題に対して、音符の開始時刻を前へ移動すると、通信層でそろえた拍位置そのものを変えてしまう。そこで拍位置は変更せず、`BRASS_ATTACK_SCALE = 0.45` として音色 JSON の `attack` だけを短縮した。確認用スケッチでは全パート再生と単独再生を切り替え、`MASTER_GAIN`、各パートの音量、`DRUM_AMPLITUDE` も聴き比べながら調整した。ただし、これらの値は確認環境での聴感調整であり、発表会場と同じスピーカを用いた統制評価によって決定した値ではない。

5.3 確認用スケッチの処理フロー

図 2 に、Processing 確認用スケッチの処理の流れを示す。起動時に音色 JSON とドラム楽譜を準備した後、キー入力があるまで待機する。再生時は、対象パートの `ScoreEvent` を順に走査し、休符以外のイベントについて開始拍、音量、移調後の音高を求める。その後、楽器定義に応じて原音サンプル、ドラムサンプル、または倍音加算合成を選択し、Minim の `playNote()` へ登録する。

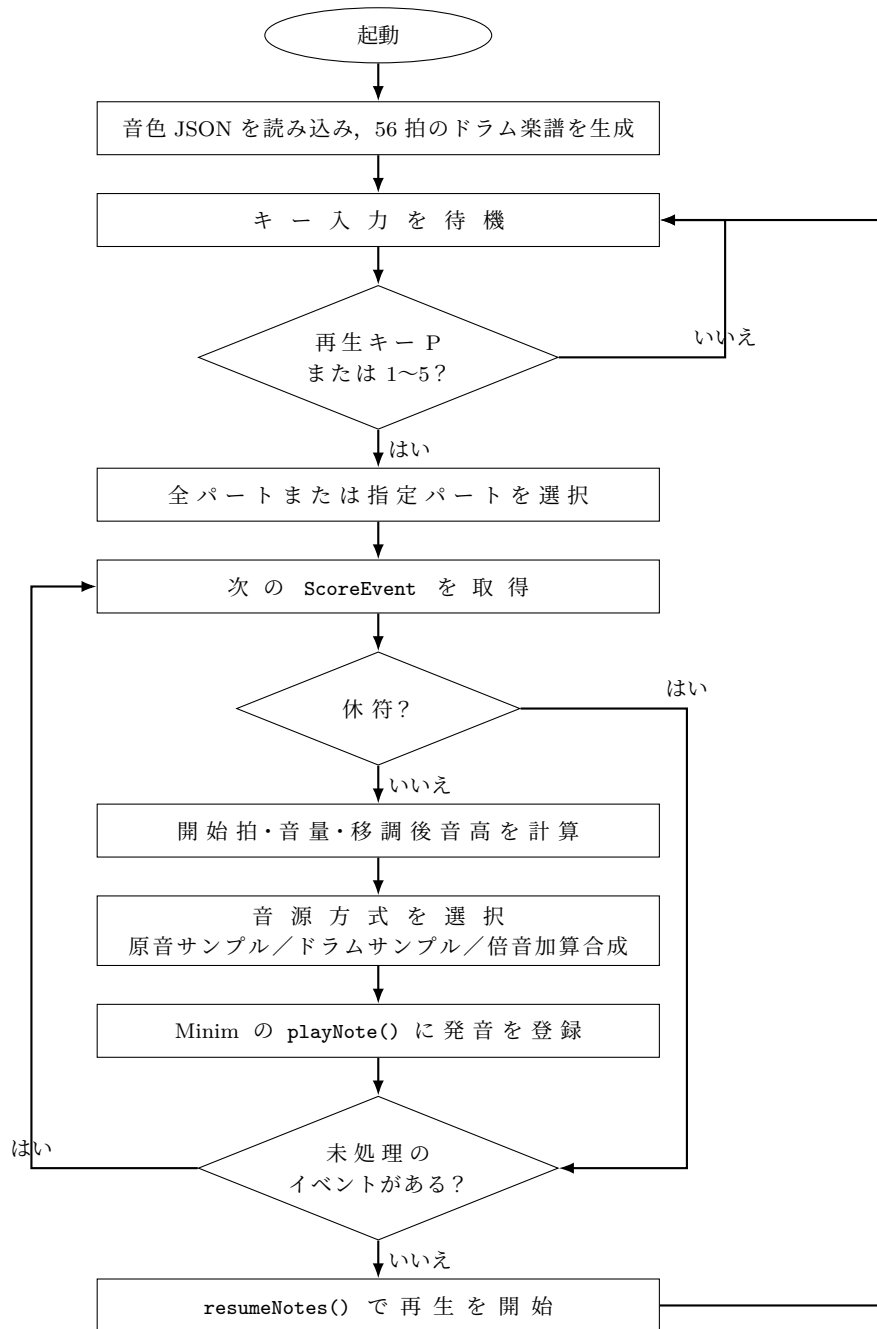


図 2 Processing 確認用スケッチの処理フロー

6 評価と考察

6.1 評価方法

実機評価では、指揮者 ESP32-S3 DevKitC 1 台、楽器 Arduino UNO R4 WiFi 5 台、Processing PC 5 台を用いた。各評価項目では、MOP_TEST コンパイルフラグによってノード側のシリアルログを出力し、Python スクリプトで CSV への記録と判定を行った。楽器間同期誤差については、USB シリアルの到着時刻ではなく、各楽器ノードが記録した推定マスタ時刻を比較した。これにより、PC 側のシリアル転送遅延が同期誤差の計測に混入することを避けた [5]。

表 3 に主な評価結果を示す。

表 3 実機評価の主な結果

項目	目標	実測値	判定
拍検出	正確性 90%以上	120 拍 中 120 拍 を 検 出 (100.0%)	達成
楽譜再生	誤ノート 0 件	誤ノート 0 件	達成
楽器間同期	同期誤差 20 ms 以下	中央値 7 ms, 平均 10.8 ms, p95 48.6 ms, 最大 65 ms	条件付き受容
音階誤差 (合成 再現)	平均 3.6 cent 未満	平均絶対誤差 0.737 cent, 最大 1.907 cent	参考値
発音予約の効果	予定時刻との差を小さく する	受信直後の発音 (仮定) 101.2 ms から予約発音 12.1 ms へ	改善
テンポ追従	2 拍以内	最大 1.2 拍	達成
起動時間	5 秒以内	最大 2.3 秒	達成
CPU 負荷	入力 2 ms 以下	最大 0.72 ms	達成
パケットロス	5%以下	全楽器ノード 0.0%	達成

拍検出では、120 BPM のメトロノームに合わせた手振りに対して、120 拍中 120 拍を検出した。テンポを 80 BPM から 130 BPM へ変化させた試験では、計測できた 4 台の楽器ノードが同じ変化に追従し、最も遅い node_04 でも 1.2 拍以内に収まった [5]。図 3 に、各ノードが受信した BPM の推移を示す。この試験では node_02 のデータを Wi-Fi 接続不良により取得できなかったため、残る 4 台を評価対象とした。後日の試験では、5 台の楽器ノードでパケットの欠落はなく、楽譜データとの誤ノートも 0 件であった。これらの結果から、筆者が作成した楽譜データを含む演奏制御は、実機構成でも意図した拍と音高を維持できたと考える。

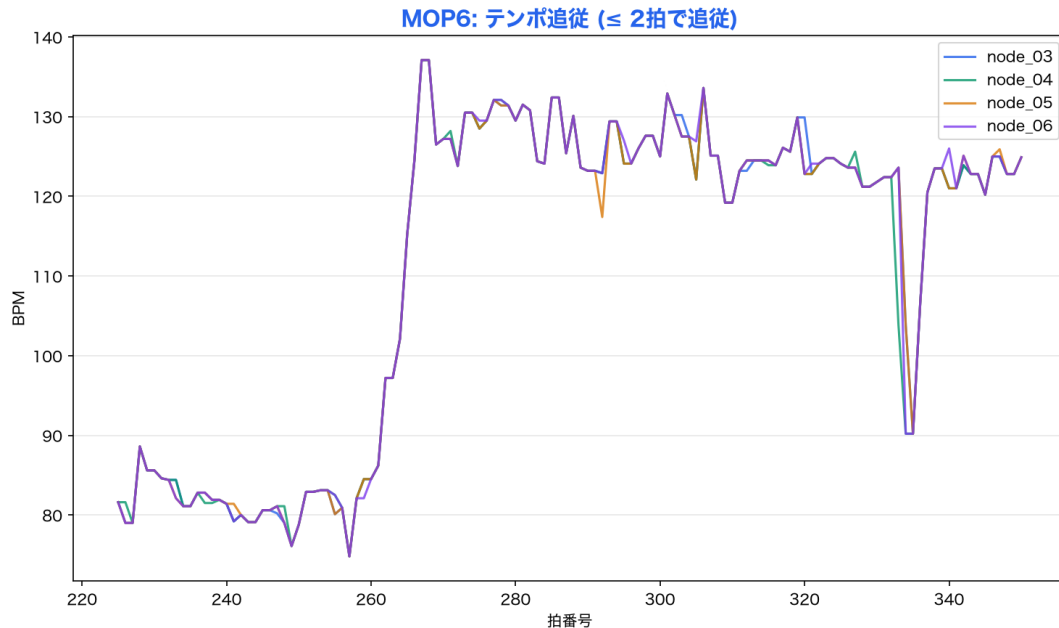


図3 80 BPM から 130 BPM への変化に対する楽器ノードのテンポ追従

6.2 同期と通信遅延の考察

最終構成では、173 拍を 5 台の楽器ノードで計測し、楽器間同期誤差の中央値は 7 ms、平均は 10.8 ms であった。173 拍中 157 拍 (90.8%) は 20 ms 以内に収まっており、通常の拍では目標値より十分に小さいずれで演奏できた。一方で、p95 は 48.6 ms、最大値は 65 ms となり、最大値が 20 ms 以下という厳密な目標は満たしていない。したがって、本報告書では同期性能を無条件に達成とせず、実演上は受容できたが、外れ値を残す結果として扱う。図 4 に、最終構成における各拍の最速ノードと最遅ノードの発音時刻差を示す。グラフからも、多くの拍が 20 ms 以内に収まる一方、16 拍で閾値を超えていることが分かる [5]。

調査の結果、SoftAP の UDP マルチキャストでは約 204.8 ms 周期でパケットがまとまって配送される現象が確認された。そこで指揮者ノードは、拍情報に 220 ms 先の発音予定時刻を付けて送信し、楽器ノードは時計同期した時刻で予定時刻まで待機して発音する方式に変更した。この変更により、発音予約に間に合わなかった受信の割合は、対策前の 45.4% から最終構成では 3.1% まで低下した。生の片道通信遅延はこのネットワーク構成では小さく安定させられないため、予約した時刻に発音できるかを評価指標にする方が、システムの目的に適している。

発表用資料では、この予約方式の効果を、予定発音時刻との差として比較している。最終構成のログから、受信した瞬間に発音したと仮定した場合の差は平均 101.2 ms であったのに対し、予約時刻まで待って実際に発音した場合の発火遅れは平均 12.1 ms であった。前者は受信時点と予定時刻との差から求めた仮想的な値であり、生の片道通信遅延ではない。この比較は通信のばらつきを予約発音で吸収できたことを示す MOP5 の評価であり、楽器同士の発音時刻差を扱う MOP4 の同

期誤差とは区別して解釈する必要がある。

ただし、この方式では指揮情報の検出から発音までに約 220 ms 程度の一定の時間差を設ける必要がある。質疑応答では、実際の指揮は演奏者が予備動作を見てから発音するため、指揮棒の動きと楽器音を完全に同時にするのではなく、その時間関係を意図的に設計すべきであるとの指摘を受けた。これは通信による不規則な遅れを許容するものではない。現在の 220 ms は通信のばらつきを吸収するための予約時間であり、指揮の予備動作と振り下ろしを音の発音時刻に対応付けた設計にはなっていない。今後は、予備動作の段階で発音予定時刻を予約し、振り下ろしと発音が対応するように、意図した先行時間を設計する必要がある。また、周期的なデバイス内部処理の停止により、一部の拍で 30~60 ms 程度の発音遅れが生じることが分かった。今後は WiFi モジュール由来と考えられる停止の原因を調査し、このような不規則な外れ値を減らす必要がある。

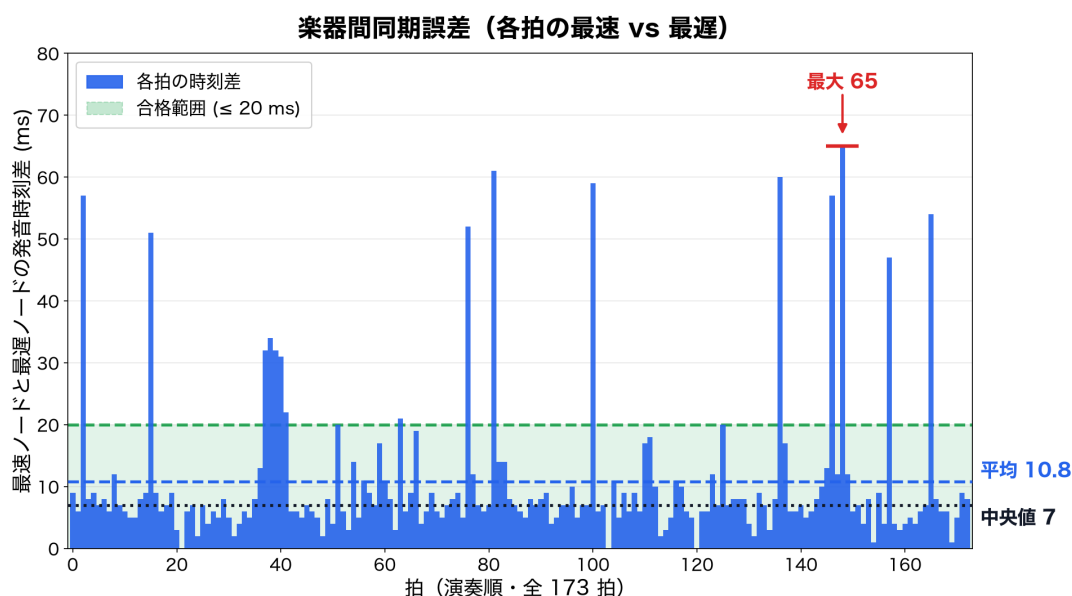


図 4 最終構成における楽器間同期誤差 (173 拍)

6.3 音色と楽譜に関する考察

Processing 上では、トランペット、ホルン、フルート、オルガンによる 4 声輪唱とドラム伴奏を再生できた。4 声は 0, 8, 16, 24 拍で順に入り、ドラムは 56 拍ぶん用意したため、最後のパートが終わるまで伴奏を維持できる。また、MASTER_GAIN = 1.35, DRUM_AMPLITUDE = 0.095, FLUTE_SAMPLE_GAIN = 1.32 などを調整し、旋律とドラムの音量バランスを改善した。

音階誤差の参考評価では、最終システムで使用する 4 種類の旋律楽器について、楽譜で用いる C, D, E, F, G, A の 6 音を対象にした。Processing の加算合成処理を Python で忠実に再現し、平均律の理論周波数と比較した結果、24 音の平均絶対誤差は 0.737 cent、最大誤差はホルンの 1.907 cent であった [6]。図 5 に、成果発表用に作成した楽器別の平均絶対誤差を示す [7]。計算上は目標の平均 3.6 cent 未満を満たし、4 楽器すべてが目標範囲内であった。

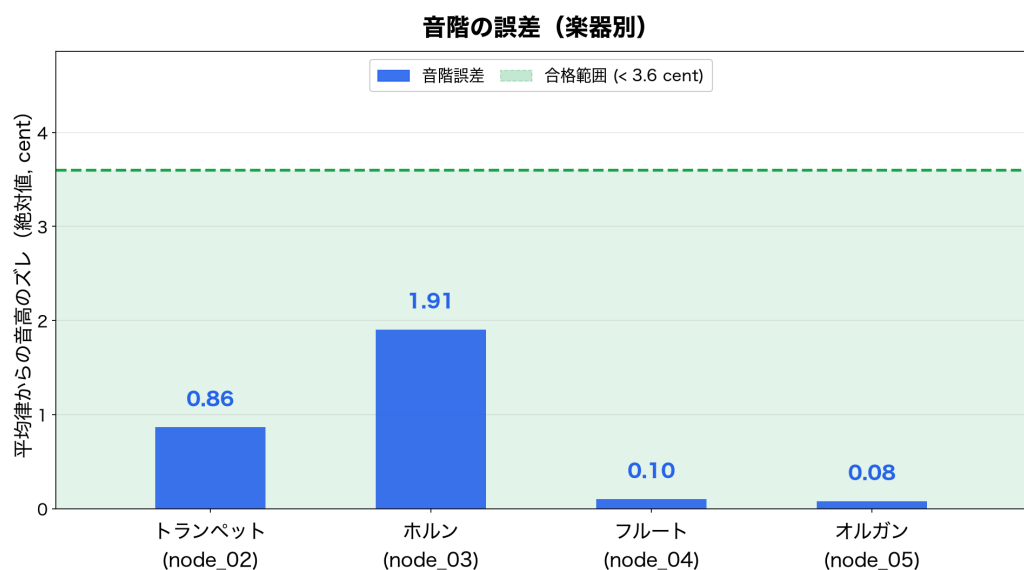


図 5 合成処理を再現した楽器別の平均音階誤差

表 4 観客アンケートによるエンターテインメント性評価 ($n = 43$)

評価項目	平均	中央値	6 点以上
没入感・臨場感	6.09	6	32 件 (74.4%)
一体感・同期の心地よさ	6.07	6	32 件 (74.4%)
演出の華やかさ	5.14	5	16 件 (37.2%)
楽しさ・高揚感	6.21	7	33 件 (76.7%)

ただし、この評価は音色 JSON と合成式に由来する音高を対象にしたシミュレーションであり、実際のスピーカ出力を録音して周波数分析する本来の MOP2 は未検証である。したがって、表 3 では達成判定に用いず、合成処理上の参考値として扱う。また、音階誤差は音高の正確さを示す指標であり、目的の楽器らしく聞こえるかを直接示すものではない。Minim の内部処理、サウンドカードの D/A 変換、出力機器に起因する音高への影響と、楽器音らしさの聴取評価は、今後の実音録音・被験者評価で確認する必要がある。

筆者の担当範囲に限ると、主な成果は、楽譜を共通化しながら開始拍と音域を変える構成にしたことである。この構成は、楽譜の修正箇所を減らし、4 声の入り方を比較しやすくする利点がある。また、音色 JSON をスケッチ内の data/フォルダに同梱したため、外部フォルダに依存せず再生確認できる。今後は、実際の発表会場やスピーカ環境で音色と音量を確認するとともに、録音した音声の周波数分析によって MOP2 の結果を確認する必要がある。

6.4 観客アンケートによるエンターテインメント性評価

講義内のシステム発表会において、チーム 23 を評価対象として得られた 43 件の回答を集計した [8]。アンケートでは、没入感・臨場感、一体感・同期の心地よさ、演出の華やかさ、楽しさ・高揚感の 4 項目を、7 点満点で評価した。表 4 に結果を示す。

楽しさ・高揚感は 4 項目中で最も高く、中央値も 7 点であった。また、没入感・臨場感および一

体感・同期の心地よさも平均 6 点を上回り、いずれも回答者の 7 割以上が 6 点以上と評価した。自由記述には、指揮棒でテンポを変更できる点や、実際に指揮者を体験できる点を肯定的に評価する意見があった。このことから、指揮ジェスチャを演奏制御に結び付けた設計は、発表会の観客にとって楽しさと没入感につながったと考えられる。

一方、演出の華やかさは平均 5.14 点で他の項目より低かった。本アンケートは講義内の発表会で得た回答であり、対象者や環境を統制した比較実験ではないため、結果を一般化することはできない。しかし、視覚演出や舞台上での見せ方を改善することが、今後のエンターテインメント性向上の課題として示された。

7 開発支援ツールの利用

本プロジェクトでは、生成 AI として OpenAI Codex を利用し、設計資料の整理、Processing スケッチの修正方針検討、README と作業ログの記述補助に用いた。筆者の担当では、4 声輪唱スケッチへのフルート・オルガン追加、tone_sample による原音ループ再生、ドラムと旋律の音量調整について相談した。Codex からは、tone_sample を読み込む ToneSampleUGen、FLUTE_SAMPLE_GAIN、DRUM_AMPLITUDE などの実装・調整案が提示された [4]。

出力の検証では、スケッチの MELODY_PARTS と定数、音色 JSON の tone_sample、README、Git 履歴を照合した。その結果、フルート追加前の提案値 DRUM_AMPLITUDE = 0.11 は、楽器差し替え後の音量バランスに合わせて 0.095 に修正した。また、確認用スケッチの楽器構成と本番の最終構成を区別し、本報告書のシステム全体説明には成果発表資料で確認した最終構成を採用した。検証を実施していない長時間再生と発表会場での聴取評価は、AI の記述から事実として採用せず、未実施項目として作業ログと本文に残した。

8 おわりに

本報告書では、チーム 23 の「タクトーン」について、システム全体の構成と筆者の担当した楽譜データ・Processing 確認用スケッチを中心に述べた。本システムは、指揮者ノードの IMU ジェスチャを使って演奏を制御し、複数の楽器ノードと PC 上の音色合成を組み合わせることで、指揮を振る体験を演奏に結び付けることを目指したものである。これにより、複数の実演奏者を集めることが難しい場合でも、一人で指揮と合奏の関係を体験できる環境を実現した。

筆者は、「かえるのうた」を 4 声輪唱として確認できる Processing スケッチを作成し、開始拍、音域、音色、ドラム伴奏、音量バランスを調整した。実機評価では、拍検出 100.0%、誤ノート 0 件、テンポ追従最大 1.2 拍、パケットロス 0.0%を確認できた。また、合成処理を再現した音階評価では、平均絶対誤差 0.737 cent となり、計算上は目標の 3.6 cent 未満であった。発表会の観客アンケートでは、楽しさ・高揚感が平均 6.21 点、没入感・臨場感が平均 6.09 点となり、指揮を振って演奏に参加する体験が高く評価された。一方で、同期誤差には最大 65 ms の外れ値が残り、音響出力を録音して評価する本来の音階誤差と、楽器音らしさの聴取評価は未実施である。今後は、これ

らの課題に加え、視覚演出や舞台上での見せ方も改善し、より安定して楽しめる演奏体験につなげる。

参考文献

- [1] ハッカソン1 授業資料, Hackathon1-260408 版.pdf, 2026 年.
- [2] ハッカソン1 報告書評価資料, Hackathon1-Ruburic2026.pdf, 2026 年.
- [3] チーム 23, システム概要・通信・同期に関する設計資料, docs/src/content/docs/system/, 2026 年.
- [4] 齋藤翔太, week9～week11 作業ログおよび Processing 確認用スケッチ, work/saito/week9/, work/saito/week10/, work/saito/week11/, 2026 年.
- [5] チーム 23, 「MOP 検証レポート (最終)」, tools/verification/results/MOP_REPORT_20260711.md, 2026 年 7 月 11 日.
- [6] チーム 23, 「MOP2: 音階の誤差 評価記録」, tools/verification/results/mop2/evaluation.md, 2026 年 7 月 12 日.
- [7] チーム 23, 「タクトーン 成果発表会スライド」および MOP2・MOP4・MOP5 発表用グラフ, 2026 年 7 月.
- [8] 講義内システム発表会, 「Arduino オーケストラに対するエンターテインメント性評価実験」回答データ, チーム 23 の有効回答 43 件, 2026 年 7 月 8 日.
- [9] 文化庁, 文化芸術推進基本計画 (第 2 期) - 価値創造と社会・経済の活性化 -, 2023 年, https://www.bunka.go.jp/seisaku/bunka_gyosei/hoshin/pdf/93856401_01.pdf.
- [10] 総務省統計局, 令和 3 年社会生活基本調査 結果の概要, 2022 年, <https://www.stat.go.jp/info/kenkyu/roudou/r5/pdf/20siryou3.pdf>.
- [11] ヤマハ株式会社, SYNCROOM について, <https://syncroom.yamaha.com/about/> (2026 年 7 月 16 日閲覧).
- [12] 宇佐聡史, 持田康典, 「マルチモーダル指揮シミュレータ」, 日本ファジィ学会誌, 第 10 巻, 第 4 号, pp.707-716, 1998 年, https://doi.org/10.3156/jfuzzy.10.4_127.
- [13] 新山王政和, 「指揮基本動作における初心者と熟達者の動作タイミングの違いに関する分析的研究」, 音楽教育学, 第 34 巻, 第 2 号, pp.1-12, 2004 年, https://doi.org/10.20614/jjomer.34.2_1.
- [14] 河瀬諭, 「合奏における演奏者間コミュニケーション-タイミング調整とその手がかり-」, 心理学評論, 第 57 巻, 第 4 号, pp.495-510, 2014 年, https://doi.org/10.24602/sjpr.57.4_495.

A 役割分担とスケジュール

表 A.1 に、設計資料に基づく主な役割分担を示す。

表 A.1 主な役割分担	
メンバー	主な担当
塩澤 匠生	Arduino 系プログラム全般、指揮者ノード、楽器ノード、共通層
齋藤 翔太	楽譜データ作成、音階生成ロジック検討、Processing 確認用スケッチ
梅澤 颯太	Processing による音色合成、演奏再生
御代川 稜	議事録、実機検証、発表準備補助
地曳 賢人	議事録、実機検証、発表準備補助
片岡 聖	議事録、実機検証、発表準備補助

図 A.1 に、授業スケジュールをもとにした概略スケジュールを示す。

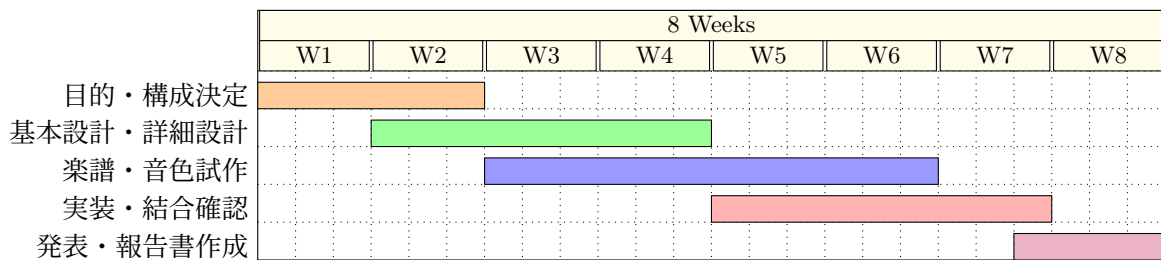


図 A.1 概略スケジュール

B プログラムソース

以下に、筆者が実機結合前の楽譜・輪唱・ドラム伴奏・音色の確認に使用した Processing スケッチの全体を示す。掲載コードは、work/saito/week10/kaeru_score_week10_adjusted/kaeru_score_week10_adjusted.pde の 2026 年 7 月 8 日時点の内容を基に、処理は変更せず、付録に不要な作業経緯と重複説明のコメントのみを整理したものである。

このスケッチは本番用 Processing アプリではなく、筆者の担当範囲を独立して確認するための環境である。また、作成時点のフルート、トランペット、トロンボーン、オルガンという 4 声構成を保持しており、表 2 に示した本番の最終構成とは用途が異なる。音色 JSON と原音サンプルはプログラムソースではないため掲載せず、リポジトリ内の data/ に保存している。

リスト 1 Processing 確認用スケッチ（全体）

```

1 import ddf.minim.*;
2 import ddf.minim.ugens.*;
3
4 final int BPM = 96;
5 final int NOTE_ON = 0x01;
6 final int REST = 0x04;
7 final int MAX_HARMONICS = 12;

```

```

8 | final int KICK_DRUM = 36;
9 | final int SNARE_DRUM = 38;
10 | final int CLOSED_HI_HAT = 42;
11 | final int CRASH_CYMBAL = 49;
12 | final float MASTER_GAIN = 1.35f;
13 | // 拍位置は変えず、金管の音の立ち上がりだけを速くしてドラムとの聴感上のズレを減らす。
14 | final float BRASS_ATTACK_SCALE = 0.45f;
15 | final float RECORDED_DRUM_GAIN = 2.6f;
16 | final float FLUTE_SAMPLE_GAIN = 1.32f;
17 | final String SOURCE_DATA_DIRECTORY = "data/";
18 |
19 | final String[] DRUM_INSTRUMENT_FILES = {
20 |     "kick.tweaked.instrument.json",
21 |     "snare.tweaked.instrument.json",
22 |     "Hi-hat.tweaked.instrument.json",
23 |     "crash.tweaked.instrument.json"
24 | };
25 |
26 | Minim minim;
27 | AudioOutput out;
28 | TimbreData[] brassTimbres;
29 | TimbreData[] drumTimbres;
30 | AudioSample[] recordedDrumSamples;
31 | ScoreEvent[] drumScore;
32 | String currentMode = "P キーで4声の主旋律と4/4ドラムを再生します";
33 |
34 | class ScoreEvent {
35 |     int beatAt;
36 |     int noteNumber;
37 |     int velocity;
38 |     int durationQ8;
39 |     int flags;
40 |     int subNote;
41 |     int subVelocity;
42 |     int subOffsetQ8;
43 |     int subDurationQ8;
44 |
45 |     ScoreEvent(int beatAt, int noteNumber, int velocity, int durationQ8, int flags) {
46 |         this(beatAt, noteNumber, velocity, durationQ8, flags, 0, 0, 0, 0);
47 |     }
48 |
49 |     ScoreEvent(int beatAt, int noteNumber, int velocity, int durationQ8, int flags,
50 |         int subNote, int subVelocity, int subOffsetQ8, int subDurationQ8) {
51 |         this.beatAt = beatAt;
52 |         this.noteNumber = noteNumber;
53 |         this.velocity = velocity;
54 |         this.durationQ8 = durationQ8;
55 |         this.flags = flags;
56 |         this.subNote = subNote;
57 |         this.subVelocity = subVelocity;
58 |         this.subOffsetQ8 = subOffsetQ8;
59 |         this.subDurationQ8 = subDurationQ8;
60 |     }
61 |
62 |     boolean isRest() {
63 |         return noteNumber == 0 || (flags & REST) != 0;
64 |     }
65 | }
66 |
67 | class PartDefinition {
68 |     String name;
69 |     String instrumentFile;
70 |     int startBeat;
71 |     int octaveShift;
72 |     float amplitude;
73 |
74 |     PartDefinition(String name, String instrumentFile, int startBeat, int octaveShift, float amplitude) {
75 |         this.name = name;
76 |         this.instrumentFile = instrumentFile;
77 |         this.startBeat = startBeat;
78 |         this.octaveShift = octaveShift;
79 |         this.amplitude = amplitude;
80 |     }
81 | }
82 |
83 | final PartDefinition[] MELODY_PARTS = {
84 |     new PartDefinition("主旋律1 / フルート", "flute.tweaked.instrument.json", 0, 12, 0.28f),
85 |     new PartDefinition("主旋律2 / トランペット", "trumpets.tweaked.instrument.json", 8, 12, 0.22f),
86 |     new PartDefinition("主旋律3 / トロンボーン", "trombones.tweaked.instrument.json", 16, -12, 0.22f),
87 |     new PartDefinition("主旋律4 / オルガン", "organ.tweaked.instrument.json", 24, -12, 0.25f)

```



```

88 };
89 final float DRUM_AMPLITUDE = 0.095f;
90
91 class TimbreData {
92     String name;
93     float fundamentalHz;
94     float[] harmonicRatios;
95     float[] harmonicGains;
96     float noiseLevel;
97     float attackSec;
98     float decaySec;
99     float sustainLevel;
100    float releaseSec;
101    float[] drumSample;
102    float drumSampleRate;
103    float[] toneSample;
104    float toneSampleRate;
105    int toneRootMidiNote;
106    int toneLoopStartSample;
107    int toneLoopEndSample;
108
109    TimbreData(String filename) {
110        JSONObject json = loadJSONObject(sketchPath(SOURCE_DATA_DIRECTORY + filename));
111        if (json == null) {
112            throw new RuntimeException("音色 JSON を読み込めません: " + filename);
113        }
114
115        name = json.getString("name");
116        fundamentalHz = json.getFloat("fundamental_hz", 440.0f);
117        JSONObject sourceEnvelope = json.getJSONObject("envelope");
118        attackSec = sourceEnvelope.getFloat("attack_sec");
119        decaySec = sourceEnvelope.getFloat("decay_sec");
120        sustainLevel = sourceEnvelope.getFloat("sustain_level");
121        releaseSec = sourceEnvelope.getFloat("release_sec");
122        noiseLevel = json.containsKey("noise") ? json.getJSONObject("noise").getFloat("level", 0) : 0;
123
124        JSONArray sourceHarmonics = json.getJSONArray("harmonics");
125        int harmonicCount = min(MAX_HARMONICS, sourceHarmonics.size());
126        harmonicRatios = new float[harmonicCount];
127        harmonicGains = new float[harmonicCount];
128        float gainSum = 0;
129        for (int i = 0; i < harmonicCount; i++) {
130            JSONObject harmonic = sourceHarmonics.getJSONObject(i);
131            harmonicRatios[i] = harmonic.getFloat("ratio");
132            harmonicGains[i] = harmonic.getFloat("amp");
133            gainSum += harmonicGains[i];
134        }
135        if (gainSum <= 0) {
136            throw new RuntimeException("倍音の振幅合計が 0 以下です: " + filename);
137        }
138        for (int i = 0; i < harmonicCount; i++) {
139            harmonicGains[i] /= gainSum;
140        }
141
142        // フルートなどの持続楽器が持つ原音本体サンプル。倍音合成より原音感を優先して鳴らす。
143        if (json.containsKey("tone_sample")) {
144            JSONObject sourceToneSample = json.getJSONObject("tone_sample");
145            JSONArray sourceValues = sourceToneSample.getJSONArray("values");
146            if (sourceValues != null && sourceValues.size() > 0) {
147                toneSample = new float[sourceValues.size()];
148                for (int i = 0; i < toneSample.length; i++) {
149                    toneSample[i] = sourceValues.getFloat(i);
150                }
151                toneSampleRate = sourceToneSample.getFloat("sample_rate", 22050.0f);
152                toneRootMidiNote = sourceToneSample.getInt("root_midi_note", json.getInt("midi_note", 60));
153                toneLoopStartSample = sourceToneSample.getInt("loop_start_sample", max(0, round(sourceToneSample.getFloat("loop_start_sec", 0) * toneSampleRate)));
154                toneLoopEndSample = sourceToneSample.getInt("loop_end_sample", min(toneSample.length, round(sourceToneSample.getFloat("loop_end_sec", toneSample.length / toneSampleRate) * toneSampleRate)));
155                toneLoopStartSample = constrain(toneLoopStartSample, 0, max(0, toneSample.length - 2));
156                toneLoopEndSample = constrain(toneLoopEndSample, toneLoopStartSample + 2, toneSample.length);
157            }
158        }
159
160        // 打楽器プロファイルが持つ原音1打。クラッシュではこの非周期波形を優先する。
161        if (json.containsKey("drum_sample")) {
162            JSONObject sourceDrumSample = json.getJSONObject("drum_sample");
163            JSONArray sourceValues = sourceDrumSample.getJSONArray("values");
164            if (sourceValues != null && sourceValues.size() > 0) {
165                drumSample = new float[sourceValues.size()];

```



```

166         for (int i = 0; i < drumSample.length; i++) {
167             drumSample[i] = sourceValues.getFloat(i);
168         }
169         drumSampleRate = sourceDrumSample.getFloat("sample_rate", 44100.0f);
170     }
171 }
172 }
173 }
174
175 // 「かえるのうた」の基準譜。C4~A4 を基準にして、各パートがオクターブ移調して演奏する。
176 ScoreEvent[] MELODY_SCORE = {
177     new ScoreEvent( 0, 60, 96, 256, NOTE_ON), new ScoreEvent( 1, 62, 96, 256, NOTE_ON),
178     new ScoreEvent( 2, 64, 96, 256, NOTE_ON), new ScoreEvent( 3, 65, 96, 256, NOTE_ON),
179     new ScoreEvent( 4, 64, 96, 256, NOTE_ON), new ScoreEvent( 5, 62, 96, 256, NOTE_ON),
180     new ScoreEvent( 6, 60, 96, 512, NOTE_ON), new ScoreEvent( 7, 0, 0, 256, REST),
181     new ScoreEvent( 8, 64, 92, 256, NOTE_ON), new ScoreEvent( 9, 65, 92, 256, NOTE_ON),
182     new ScoreEvent(10, 67, 92, 256, NOTE_ON), new ScoreEvent(11, 69, 92, 256, NOTE_ON),
183     new ScoreEvent(12, 67, 92, 256, NOTE_ON), new ScoreEvent(13, 65, 92, 256, NOTE_ON),
184     new ScoreEvent(14, 64, 96, 512, NOTE_ON), new ScoreEvent(15, 0, 0, 256, REST),
185     new ScoreEvent(16, 60, 90, 256, NOTE_ON), new ScoreEvent(17, 0, 0, 256, REST),
186     new ScoreEvent(18, 60, 90, 256, NOTE_ON), new ScoreEvent(19, 0, 0, 256, REST),
187     new ScoreEvent(20, 60, 90, 256, NOTE_ON), new ScoreEvent(21, 0, 0, 256, REST),
188     new ScoreEvent(22, 60, 90, 256, NOTE_ON), new ScoreEvent(23, 0, 0, 256, REST),
189     new ScoreEvent(24, 60, 96, 128, NOTE_ON, 60, 96, 128, 128),
190     new ScoreEvent(25, 62, 96, 128, NOTE_ON, 62, 96, 128, 128),
191     new ScoreEvent(26, 64, 96, 128, NOTE_ON, 64, 96, 128, 128),
192     new ScoreEvent(27, 65, 96, 128, NOTE_ON, 65, 96, 128, 128),
193     new ScoreEvent(28, 64, 96, 256, NOTE_ON), new ScoreEvent(29, 62, 96, 256, NOTE_ON),
194     new ScoreEvent(30, 60, 100, 512, NOTE_ON), new ScoreEvent(31, 0, 0, 256, REST)
195 };
196
197 // 56拍: 24拍遅れの最後の声部が終わるまで伴奏する。
198 // 各小節の1・3拍目をキック、2・4拍目をスネアにする。
199 // 各声部の入り (0/8/16/24拍) と最後の拍だけクラッシュ+キックで強調する。
200 ScoreEvent[] createDrumScore() {
201     ScoreEvent[] score = new ScoreEvent[56];
202     for (int beat = 0; beat < score.length; beat++) {
203         int beatInBar = beat % 4;
204         boolean strongBeat = beatInBar == 0;
205         boolean kickBeat = beatInBar == 0 || beatInBar == 2;
206         int note = kickBeat ? KICK_DRUM : SNARE_DRUM;
207         int velocity = kickBeat ? (strongBeat ? 86 : 78) : 74;
208
209         if (beat == 0 || beat == 8 || beat == 16 || beat == 24 || beat == 55) {
210             int crashVelocity = beat == 55 ? 92 : 84;
211             int kickVelocity = beat == 55 ? 86 : 82;
212             score[beat] = new ScoreEvent(beat, CRASH_CYMBAL, crashVelocity, 256, NOTE_ON,
213                                         KICK_DRUM, kickVelocity, 0, 256);
214             continue;
215         }
216
217         score[beat] = new ScoreEvent(beat, note, velocity, 256, NOTE_ON);
218     }
219     return score;
220 }
221
222 class BrassNote implements Instrument {
223     Summer mix;
224     ADSR envelope;
225
226     BrassNote(float frequency, float amplitude, TimbreData timbre) {
227         mix = new Summer();
228         for (int i = 0; i < timbre.harmonicRatios.length; i++) {
229             Oscil harmonic = new Oscil(frequency * timbre.harmonicRatios[i],
230                                         amplitude * timbre.harmonicGains[i], Waves.SINE);
231             harmonic.patch(mix);
232         }
233         float attack = max(0.003f, timbre.attackSec * BRASS_ATTACK_SCALE);
234         envelope = new ADSR(1.0f, attack, timbre.decaySec,
235                             timbre.sustainLevel, timbre.releaseSec);
236         mix.patch(envelope);
237     }
238
239     void noteOn(float duration) { envelope.noteOn(); envelope.patch(out); }
240     void noteOff() { envelope.unpatchAfterRelease(out); envelope.noteOff(); }
241 }
242
243 class ToneSampleUGen extends UGen {
244     float[] sample;
245     float sourceSampleRate;

```

```

246 float position;
247 float step;
248 float pitchRatio;
249 float lastOutputSampleRate;
250 int loopStart;
251 int loopEnd;
252
253 ToneSampleUGen(TimbreData timbre, int midiNote) {
254     sample = timbre.toneSample;
255     sourceSampleRate = timbre.toneSampleRate;
256     loopStart = timbre.toneLoopStartSample;
257     loopEnd = timbre.toneLoopEndSample;
258     position = 0;
259     pitchRatio = pow(2.0f, (midiNote - timbre.toneRootMidiNote) / 12.0f);
260     lastOutputSampleRate = 0;
261 }
262
263 protected void uGenerate(float[] channels) {
264     if (sample == null || sample.length == 0) {
265         for (int i = 0; i < channels.length; i++) channels[i] = 0;
266         return;
267     }
268     float outputSampleRate = max(1.0f, sampleRate());
269     if (outputSampleRate != lastOutputSampleRate) {
270         step = pitchRatio * sourceSampleRate / outputSampleRate;
271         lastOutputSampleRate = outputSampleRate;
272     }
273
274     if (position >= loopEnd) {
275         float loopLength = max(1, loopEnd - loopStart);
276         position = loopStart + ((position - loopStart) % loopLength);
277     }
278
279     int i0 = constrain(floor(position), 0, sample.length - 1);
280     int i1 = min(i0 + 1, sample.length - 1);
281     float frac = position - i0;
282     float value = lerp(sample[i0], sample[i1], frac);
283     position += step;
284     for (int i = 0; i < channels.length; i++) channels[i] = value;
285 }
286 }
287
288 class SampledMelodyNote implements Instrument {
289     ToneSampleUGen tone;
290     ADSR envelope;
291
292     SampledMelodyNote(int midiNote, float amplitude, TimbreData timbre) {
293         tone = new ToneSampleUGen(timbre, midiNote);
294         envelope = new ADSR(constrain(amplitude * FLUTE_SAMPLE_GAIN, 0.0f, 1.0f),
295                             0.004f, 0.04f, 0.92f, max(0.08f, timbre.releaseSec * 0.45f));
296         tone.patch(envelope);
297     }
298
299     void noteOn(float duration) { envelope.noteOn(); envelope.patch(out); }
300     void noteOff() { envelope.unpatchAfterRelease(out); envelope.noteOff(); }
301 }
302
303 class DrumNote implements Instrument {
304     Summer mix;
305     Noise noise;
306     ADSR envelope;
307
308     DrumNote(int noteNumber, float amplitude) {
309         mix = new Summer();
310         TimbreData timbre = drumTimbres[drumIndex(noteNumber)];
311         for (int i = 0; i < timbre.harmonicRatios.length; i++) {
312             Oscil harmonic = new Oscil(timbre.fundamentalHz * timbre.harmonicRatios[i],
313                                         amplitude * timbre.harmonicGains[i], Waves.SINE);
314             harmonic.patch(mix);
315         }
316         float noiseAmplitude = amplitude * timbre.noiseLevel * drumNoiseScale(noteNumber);
317         if (noiseAmplitude > 0.001f) {
318             noise = new Noise(noiseAmplitude, Noise.Tint.WHITE);
319             noise.patch(mix);
320         }
321         envelope = new ADSR(1.0f, timbre.attackSec, timbre.decaySec,
322                             timbre.sustainLevel, timbre.releaseSec);
323         mix.patch(envelope);
324     }
325 }

```

```

326 void noteOn(float duration) { envelope.noteOn(); envelope.patch(out); }
327 void noteOff() { envelope.unpatchAfterRelease(out); envelope.noteOff(); }
328 }
329
330 // ドラムJSONに原音1打があれば優先して再生する。サンプルがないJSONは従来どおり合成する。
331 class RecordedDrumNote implements Instrument {
332     AudioSample sample;
333     float amplitude;
334
335     RecordedDrumNote(AudioSample sample, float amplitude) {
336         this.sample = sample;
337         this.amplitude = amplitude;
338     }
339
340     void noteOn(float duration) {
341         sample.setGain(linearToDecibels(constrain(amplitude * RECORDED_DRUM_GAIN, 0.001f, 1.0f)));
342         sample.trigger();
343     }
344
345     // 原音に含まれる自然な減衰を最後まで鳴らすため、譜面上の短い長さでは停止しない。
346     void noteOff() {
347     }
348 }
349
350 void setup() {
351     size(940, 490);
352     minim = new Minim(this);
353     out = minim.getLineOut(Minim.STEREO, 1024);
354     out.setTempo(BPM);
355     brassTimbres = new TimbreData[MELODY_PARTS.length];
356     for (int i = 0; i < MELODY_PARTS.length; i++) brassTimbres[i] = new TimbreData(MELODY_PARTS[i].instrumentFile);
357     drumTimbres = new TimbreData[DRUM_INSTRUMENT_FILES.length];
358     for (int i = 0; i < DRUM_INSTRUMENT_FILES.length; i++) drumTimbres[i] = new TimbreData(DRUM_INSTRUMENT_FILES[i]);
359     recordedDrumSamples = new AudioSample[drumTimbres.length];
360     for (int i = 0; i < drumTimbres.length; i++) recordedDrumSamples[i] = createRecordedDrumSample(drumTimbres[i]);
361     drumScore = createDrumScore();
362     textFont(createFont("SansSerif", 16));
363 }
364
365 void draw() {
366     background(248, 246, 242);
367     fill(34);
368     textSize(26);
369     text("かえるのうた 4声輪唱・week10 フルート/オルガン版", 28, 42);
370     textSize(15);
371     text("固定テンポ: " + BPM + " BPM 共通主旋律: " + MELODY_SCORE.length + " 拍 ドラム: " + drumScore.length + " 拍
        全体音量倍率: x" + nf(MASTER_GAIN, 1, 2), 28, 72);
372     text("P: 全パート再生 1-5: 単独再生 再生完了後に次のキーを押してください", 28, 98);
373     for (int part = 0; part < MELODY_PARTS.length; part++) {
374         PartDefinition definition = MELODY_PARTS[part];
375         int y = 146 + part * 42;
376         fill(34);
377         text(definition.name + " 開始拍 = " + definition.startBeat + " オクターブ = " + octaveLabel(definition.
            octaveShift), 28, y);
378         fill(70 + part * 24, 130, 180);
379         rect(410 + definition.startBeat * 7, y - 14, MELODY_SCORE.length * 7, 14, 3);
380     }
381     fill(34);
382     text("リズム / ドラム 開始拍 = 0 4/4 = 1・3拍キック / 2・4拍スネア", 28, 314);
383     fill(176, 112, 90);
384     rect(410, 300, drumScore.length * 7, 14, 3);
385     fill(34);
386     text(currentMode, 28, 368);
387     textSize(13);
388     text("共通譜をオクターブ移調: フルート C5〜A5 / トランペット C5〜A5 / トロンボーン C3〜A3 / オルガン C3〜A3", 28, 410);
389     text("ドラム: 56拍の4/4パターン。声部の入りと最後だけクラッシュ+キックで強調", 28, 434);
390     text("音色 JSON はこのスケッチの data/ フォルダを参照", 28, 456);
391 }
392
393 void keyPressed() {
394     if (key == 'p' || key == 'P') playAllParts();
395     else if (key >= '1' && key <= '5') playPart(key - '1');
396 }
397
398 void playAllParts() {
399     out.pauseNotes();
400     for (int part = 0; part < MELODY_PARTS.length + 1; part++) schedulePart(part);
401     out.resumeNotes();
402     currentMode = "4声の主旋律輪唱（フルート高音・オルガン低音）と、音量を上げた4/4ドラムを再生中です。";
403 }

```

```

404
405 void playPart(int part) {
406     out.pauseNotes();
407     schedulePart(part);
408     out.resumeNotes();
409     currentMode = "单独再生中: " + partName(part) + "。";
410 }
411
412 void schedulePart(int part) {
413     ScoreEvent[] score = part == MELODY_PARTS.length ? drumScore : MELODY_SCORE;
414     int startBeat = part == MELODY_PARTS.length ? 0 : MELODY_PARTS[part].startBeat;
415     for (ScoreEvent event : score) {
416         if (event.isRest()) continue;
417         float amplitude = partAmplitude(part) * event.velocity / 127.0f * MASTER_GAIN;
418         float start = startBeat + event.beatAt;
419         out.playNote(start, event.durationQ8 / 256.0f, noteInstrument(part, event.noteNumber, amplitude));
420         if (event.subNote != 0 && event.subVelocity > 0) {
421             float subAmplitude = partAmplitude(part) * event.subVelocity / 127.0f * MASTER_GAIN;
422             out.playNote(start + event.subOffsetQ8 / 256.0f, event.subDurationQ8 / 256.0f,
423                 noteInstrument(part, event.subNote, subAmplitude));
424         }
425     }
426 }
427
428 Instrument noteInstrument(int part, int noteNumber, float amplitude) {
429     if (part == MELODY_PARTS.length) {
430         AudioSample sample = recordedDrumSamples[drumIndex(noteNumber)];
431         if (sample != null) return new RecordedDrumNote(sample, amplitude);
432         return new DrumNote(noteNumber, amplitude);
433     }
434     int shiftedNote = noteNumber + MELODY_PARTS[part].octaveShift;
435     if (brassTimbres[part].toneSample != null) {
436         return new SampledMelodyNote(shiftedNote, amplitude, brassTimbres[part]);
437     }
438     return new BrassNote(midiToFrequency(shiftedNote), amplitude, brassTimbres[part]);
439 }
440
441 AudioSample createRecordedDrumSample(TimbreData timbre) {
442     if (timbre.drumSample == null || timbre.drumSample.length == 0) return null;
443     javax.sound.sampled.AudioFormat format = new javax.sound.sampled.AudioFormat(
444         timbre.drumSampleRate, 16, 1, true, true
445     );
446     return minim.createSample(timbre.drumSample, format, 1024);
447 }
448
449 float partAmplitude(int part) { return part == MELODY_PARTS.length ? DRUM_AMPLITUDE : MELODY_PARTS[part].amplitude; }
450 String partName(int part) { return part == MELODY_PARTS.length ? "リズム / ドラム" : MELODY_PARTS[part].name; }
451 String octaveLabel(int semitones) { return (semitones > 0 ? "+" : "") + (semitones / 12) + " octave"; }
452
453 int drumIndex(int noteNumber) {
454     if (noteNumber == KICK_DRUM) return 0;
455     if (noteNumber == SNARE_DRUM) return 1;
456     if (noteNumber == CLOSED_HI_HAT) return 2;
457     if (noteNumber == CRASH_CYMBAL) return 3;
458     return 2;
459 }
460
461 float drumNoiseScale(int noteNumber) {
462     if (noteNumber == KICK_DRUM) return 0.14f;
463     if (noteNumber == SNARE_DRUM) return 0.45f;
464     if (noteNumber == CRASH_CYMBAL) return 0.30f;
465     return 0.42f;
466 }
467
468 float linearToDecibels(float amplitude) { return 20.0f * log(amplitude) / log(10.0f); }
469 float midiToFrequency(int midiNote) { return 440.0f * pow(2.0f, (midiNote - 69) / 12.0f); }

```